

Fastumi使用

写在前面：

fastumi是个高带宽系统，出现大多数问题都是带宽不足的问题，记得查自己是不是usb 3.0

UMI采集没有base坐标系，所以所有东西都需要使用relative的

官方教程：[📖 FastUMI Pro使用教程](#)

环境配置

- Ubuntu 20.04
- ROS Noetic
- 所有代码可以在<https://github.com/emia7/openpi> 获取最新代码

安装硬件sdk

代码块

```
1  git clone https://github.com/FastUMIData/FastUMI_Hardware_SDK.git
2
3  cd FastUMI_Hardware_SDK
4  cd xv/scripts/
5
6  sudo -E bash install-ros1.sh ../sdk/20260312/XVSDK_focal_amd64.deb
7
8  # 这里之后会自己写出一个catkin_ws
9  # sudo -E bash install-ros1.sh
   /home/arm/FastUMI_Hardware_SDK/xv/sdk/20260312/XVSDK_focal_amd64.deb
10 cd ~/catkin_ws/
11 roslaunch xv_sdk xv_sdk.launch
12
13
14
15 # 拓展USB带宽
16 #执行完后会自动重启终端
17 sudo -E bash multi-support.sh
```

- 若使用D435i相机，则需要安装ros驱动

代码块

```
1 sudo apt-get install ros-noetic-realsense2-camera-*
```

安装monitor

代码块

```
1 conda create -n fastumi python=3.8.5
2 conda activate fastumi
3
4 git clone https://github.com/FastUMIData/FastUMI_Monitor_Tool.git
5
6 pip install -r requirements.txt
7
8 roslaunch xv_sdk xv_sdk.launch
9
10 chmod +x fastumi_monitor_menu.sh
11 bash fastumi_monitor_menu.sh
```

会出来一个菜单，可以看各种传感器参数和可视化

常用：

2 实时打印pose，观察slam情况

21 夹爪service，采集前需要打开

20 整体图像可视化，不过可以使用rqt_image_view代替

双臂

方案3是最简便的，推荐使用，如果不行用2，个人感觉数据平台不好用

采数据：

1. 打开数据采集平台后端：

代码块

```
1 cd /home/ubuntu/qiuyi/FastUMI/FastUMI_Data_Platform_Web
2 bash ./deploy.sh
3
```

2. chrome开数据采集平台前端

<http://localhost:8000/data-collector>

3. 将两个设备放到一个相对固定的位置，开启设备，连接设备，开始采集

如果报错，关闭设备重新开启或者插拔一下USB

4. 看一下图像频率能不能跑到60Hz，打出rostopic看一下

5. 开始采集

采集后数据格式为

视频为60Hz mp4，其他为txt

为了和上面内容对齐，我们转化为mp4+json

采数据2：

采集平台两个设备是分开采集和训练的，其实用不用无所谓，感觉也不如直接用rosviz录制运行

代码块

```
1
2 cd ~/catkin_ws/
3
4 roslaunch xv_sdk xv_sdk.launch
5 # 启动fastumi
```

两个分别打开monitor来看

代码块

```
1 cd ./FastUMI_Monitor_Tool/
2
3 bash fastumi_monitor_menu.sh
4
5 21
6
7 20
```

开第三视角的D435

代码块

```
1 # 开D435
```

```
2
3  roslaunch realsense2_camera rs_camera.launch
4
5  # 可以看 rqt_image_view 来看现在的视频
```

看topic信息

代码块

```
1  rostopic list
2
3  # 打印image topic的频率
4  rostopic hz xxx
5  # 注意到两个相机都能跑到60Hz，录制时右手相机会掉帧，猜测是录制的带宽问题，尝试能不能直接录制10Hz内容
```

开一个脚本，订阅现在的几个图像信息，降采样到10Hz之后进行录制

代码块

```
1  # 把脚本放进某个 catkin 包的 scripts/（比如 FastUMI 里你已有的包）
2  chmod +x scripts/tri_image_sampler_10hz.py
3  rosrun <your_pkg> tri_image_sampler_10hz.py
4
5  # python脚本，不需要重新编译，但需要放到一个package里面，保证能rosrun出来
6  # 理论上这个脚本会得到_10hz的一系列topic
```

直接用rosbag录制

代码块

```
1  mkdir -p ~/qiuyi/FastUMI/data/bags
2  cd ~/qiuyi/FastUMI/data
3
4  rosbag record -O bags/all_10hz_$(date +%Y%m%d_%H%M%S).bag \
5    /xv_sdk/250801DR48FP25002268/color_camera/image_10hz \
6    /xv_sdk/250801DR48FP25002565/color_camera/image_10hz \
7    /camera/color/image_raw/compressed_10hz \
8    /xv_sdk/250801DR48FP25002268/slam/pose \
9    /xv_sdk/250801DR48FP25002268/clamp/Data \
10   /xv_sdk/250801DR48FP25002565/slam/pose \
11   /xv_sdk/250801DR48FP25002565/clamp/Data \
12   --lz4 --buffsize=2048 --chunksize=512
```

采数据3

启动命令太复杂了，给一个rospackage



catkin_dual.zip
251.10KB



代码块

```
1 ./start_bringup.sh
2
```

两个分别打开monitor来看

代码块

```
1 cd ./FastUMI_Monitor_Tool/
2
3 bash fastumi_monitor_menu.sh
4
5 21
6
7 2
8
9 20
```

建议把clamp和pose挂着看情况

直接用rosbag录制

代码块

```
1 mkdir -p ~/qiuyi/FastUMI/data/bags
2 cd ~/qiuyi/FastUMI/data
3
4 rosbag record -O bags/all_10hz_$(date +%Y%m%d_%H%M%S).bag \
5 /xv_sdk/250801DR48FP25002268/color_camera/image_10hz \
6 /xv_sdk/250801DR48FP25002565/color_camera/image_10hz \
7 /camera/color/image_raw/compressed_10hz \
```

```
8 /xv_sdk/250801DR48FP25002268/slam/pose \  
9 /xv_sdk/250801DR48FP25002268/clamp/Data \  
10 /xv_sdk/250801DR48FP25002565/slam/pose \  
11 /xv_sdk/250801DR48FP25002565/clamp/Data \  
12 --lz4 --buffsize=2048 --chunksize=512  
13
```

数据格式转换

1. rosbag —> mp4+json

代码块

```
1 # 有一个123的脚本和对应的bash文件  
2  
3 chmod +x batch_stage1_vis_123.sh  
4 ./batch_stage1_vis_123.sh /path/to/bags /path/to/output start_idx  
  convert_rosbag_to_mp4_vis_123.py  
5  
6 # ./umi_scripts_csw/batch_stage1_vis_123.sh  
  /home/ubuntu/qiuyi/FastUMI/data/bags  
  /home/ubuntu/qiuyi/FastUMI/data/bags/stage1 1
```

2. 查看是否有明显不对劲的state，有的话删去，对当前文件夹内容做一下sort

代码块

```
1 python3 json_sort_123.py --dir /path/to/output --start 1
```

随后将mp4和json scp过去

代码块

```
1 scp xxx.zip {your_server:/path}
```

3. 数据清洗

a. 克隆代码仓库

代码块

```
1 git clone https://github.com/emia7/openpi.git
```

```
2  cd openpi
3  git checkout dev_csw
```

b. 安装依赖

代码块

```
1  pip install numpy imageio tqdm
```

c. 检测数据质量

代码块

```
1  cd ~/openpi
2
3  python umi_scripts_csw/data_cleaning/check_trajectory_anomalies.py \
4      --data_dir /path/to/your/dataset \
5      --output_dir ./anomaly_reports
```

参数说明:

- `--data_dir`: 你的数据目录路径 (包含 MP4 + JSON)
- `--output_dir`: 检测报告输出目录

输出文件:

- `trajectory_anomaly_report.json` - 完整检测报告

d. 生成可视化报告

代码块

```
1  python umi_scripts_csw/data_cleaning/generate_visual_report.py \
2      --json_report ./anomaly_reports/trajectory_anomaly_report.json \
3      --output_dir ./anomaly_reports \
4      --format both
```

e. 筛选

代码块

```
1  ##### 策略A - 保守筛选 (只保留OK)
2  ```bash
3  python umi_scripts_csw/data_cleaning/execute_data_cleaning.py \
4      --data_dir /path/to/your/dataset \
```

```
5 --anomaly_report ./anomaly_reports/trajectory_anomaly_report.json \  
6 --severity critical \  
7 --dry_run
```

代码块

```
1 # 策略B - 标准筛选 (保留OK+MINOR)  
2 python umi_scripts_csw/data_cleaning/execute_data_cleaning.py \  
3 --data_dir /path/to/your/dataset \  
4 --anomaly_report ./anomaly_reports/trajectory_anomaly_report.json \  
5 --severity warning \  
6 --dry_run
```

预览模式 (`--dry_run`): 不实际删除, 只显示将要删除的episodes

确认预览结果无误后, 去掉 `--dry_run` 正式执行:

代码块

```
1 python umi_scripts_csw/data_cleaning/execute_data_cleaning.py \  
2 --data_dir /path/to/your/dataset \  
3 --anomaly_report ./anomaly_reports/trajectory_anomaly_report.json \  
4 --severity critical \  
5 --output_dir /path/to/your/dataset_cleaned
```

4. mp4 + json 转成lerobot

代码块

```
1 python convert_mp4_data_to_lerobot_123.py --stage1_dir={} --repo={} --task={}
```

replay

暂时挂起

训练

1. 对齐输入格式

src/openpi/policies下 xv_dual_policy

2. 改写config文件, 使用pi05_xv_dual_finetune

注意bs大小，一张卡32

3. 计算norm

代码块

```
1 uv run scripts/compute_norm_stats.py --config-name pi05_xv_dual_finetune
```

4. 开训

注意开tmux，现在的tmux和exp_name相同

代码块

```
1 tmux new -s exp_name
2
3 tmux attach -t exp_name
```

注意改exp_name

JAX会默认使用可见的所有卡，所以需要修改可见卡数为当前可用的

代码块

```
1 CUDA_VISIBLE_DEVICES=0 XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 uv run
  scripts/train.py pi05_xv_dual_finetune --exp-name={your_exp_name} --overwrite
```

注意一下loss情况，双臂要到0.01x左右

训完把norm_stats.json放在assets下

offline测试

eval_dual_relative.py

代码块

```
1 # 对比前4个时间步
2 uv run python umi_scripts_csw/eval_dual_relative.py \
3     --config=pi05_xv_dual_finetune \
4     --exp-name=your_exp \
5     --step=20000 \
```

```

6      --repo=/path/to/data \
7      --episode=0 \
8      --compare-steps="0,1,2,3"
9
10     # 对比所有时间步（假设horizon=10）
11     uv run python umi_scripts_csw/eval_dual_relative.py \
12         ... \
13         --compare-steps="all"

```

部署

与gello策略相同

openpi部分

openpi里起server，openpi的配置与前面相同，

- serve_policy 里面可以指定端口号，默认prompt等

代码块

```

1  # openpi下,注意这里指定config和路径
2  # 注意ckpt的assets下应有一个norm_stats.json
3  uv run scripts/serve_policy_dual.py policy:checkpoint --policy.config=
    {your_config_name} --policy.dir={your_policy_dir}

```

- 目前修改了wesocket_policy_server，对齐eval输出和模型输入的接口

相机发布

左臂

发布第三视角相机图像

先获取相机标识：

代码块

```

1  # realsense
2  python scripts/get_serial_num.py
3
4  # lumos
5  ls /dev/v4l/by-id/

```

代码块

```
1 # 修改serial部分
2 python scripts/camera_zmq_pub.py \
3     --name third_d455 --type realsense --serial 941322072906 \
4     --port 7000 --width 640 --height 480 --fps 30 --hz_pub 30 \
5     --out_width 420 --out_height 240 \
6     --jpeg_quality 80
```

发布腕部相机图像

代码块

```
1 # lumos
2 python scripts/camera_zmq_pub.py \
3     --name left_wrist_lumos --type lumos --serial usb-
4     XVisio_Technology_XVisio_vSLAM_250801DR48FB26001173-video-index0 \
5     --port 7001 --width 1280 --height 1280 --fps 60 --hz_pub 30 \
6     --out_width 420 --out_height 240 \
7     --jpeg_quality 80
```

代码块

```
1 # realsense, 同样更换--serial内容
2 python scripts/camera_zmq_pub.py \
3     --name left_wrist_d435 --type realsense --serial 141722078696 \
4     --port 7002 --width 640 --height 480 --fps 30 --hz_pub 30 \
5     --out_width 420 --out_height 240 \
6     --jpeg_quality 80
```

右臂

发布腕部相机图像

代码块

```
1 python scripts/camera_zmq_pub.py \
2     --name right_wrist_lumos --type lumos --serial usb-
3     XVisio_Technology_XVisio_vSLAM_250801DR48FB26001218-video-index0 \
4     --port 7003 --width 1280 --height 1280 --fps 60 --hz_pub 30 \
5     --out_width 420 --out_height 240 \
6     --jpeg_quality 80
```

代码块

```
1 python scripts/camera_zmq_pub.py \  
2     --name right_wrist_d435 --type realsense --serial 141722078696 \  
3     --port 7004 --width 640 --height 480 --fps 30 --hz_pub 30 \  
4     --out_width 420 --out_height 240 \  
5     --jpeg_quality 80
```

eval

本地跑eval，使用realRL `scripts/eval_policy.sh`

注意本地控制器需要使用relative的控制，搜索"relative" / "delta" 查看位置，进行修改

代码块

```
1 ./run_server.sh
```

代码块

```
1 # realRL, 默认使用lumos相机  
2 ./scripts/eval_policy.sh {task_name} 20  
3  
4 # --zmq_policy_image_keys=left_wrist_lumos,right_wrist_lumos,third_d455  
5 # 可以使用这个参数进行相机修改，建议写在sh里面
```

数据分析

挂起，可以参考单臂进行

nano采集

采集中用 **a / c** 播 **chirp 标音** 打点；录完整段长视频后，在本地用检测找起停时间、再切成多段

在仓库里进 `umi_scripts_csw` 再执行下述命令（把 `cd` 换成你本机路径）：

代码块

```
1 cd /path/to/umi_scripts_csw
```

1. 一次性：生成标音 WAV

改动了 `config.py` 里与**标音波形**相关的内容后，**必须**重跑：

代码块

```
1 python3 nano_sync_freq/build_assets.py
```

得到 `nano_sync_freq/assets/start.wav`、`stop.wav`。**采集中与后处理**检测只能用同一次生成结果。

2. 采集中：a 开始、c 结束 录长片

a：开始录制

c：停止录制

nano一直处在拍摄状态中，每次拍完再进行划分

使用之前检查电脑扬声器状态

代码块

```
1 python3 nano_sync_freq/play_markers.py --self-test
```

如果没问题

代码块

```
1 python3 nano_sync_freq/pedal_freq_listener.py --global
```

3. 自动划分：找峰 + 成对 + 切出 clip

代码块

```
1
2 将路径换成你的**原片**和输出目录（`out_freq` 仅为示例）：
3
4 ```bash
5 python3 nano_sync_freq/segment_by_freq_markers.py /path/to/record.mp4 \
6     --cut-dir ./out_freq \
7     --markers-out ./out_freq/markers.json
8
```

得到：

- `out_freq/clip_0000.mp4` … 一段。
- `out_freq/markers.json` — 全部分析与起停表，**要留档**（给标注、重切）。

可选项（按需加在一条命令上）：

- 只出 JSON 不切： `--write-markers-only` + `--markers-out`；或不要 `--cut-dir`。
- 不要「停音」后面的**尾长**（多录进一截停 chirp）： `--no-stop-tail`。
- 自订尾长秒数： `--stop-tail-sec 0.12` 或看 `config.py` 里 `INCLUDE_STOP_BEEP_TAIL_SEC`。

4. 人工标注校准

首跑不准时，在本地页里听原片、对表格，再导出**改过的起停表**。

推荐起小服务（`--video`、`--markers` 换成**同一原片与第 3 步的** `markers.json`）：

代码块

```
1 python3 nano_sync_freq/annotate_server.py \  
2   --video /path/to/record.mp4 \  
3   --markers /path/to/out_freq/markers.json \  
4   --port 8765
```

浏览器打开终端里给的 `http://127.0.0.1:8765/`。只本机用。

- 时间轴有**只读系统层**与**可编辑**起停；**滚轮 / 按钮**可缩放、平移。
- 在输入框**外**：**a / c** 在播放头加起/停，**Space** 播/停；**点**可编辑条竖线再按 **e** 可**起停互换**；**⌘/Ctrl+Z** 撤、**⇧+⌘/Ctrl+Z** 恢复。
- 点「**导出** `edited_markers.json`」存到如 `~/Downloads/`。

5. 重新切分

`record.mp4` 要仍是同一条；marker-in JSON 用导出的（里面一般有 `file` 等字段）：

代码块

```
1 python3 nano_sync_freq/segment_by_freq_markers.py /path/to/record.mp4 \  
2   --markers-in /path/to/edited_markers.json \  
3   --cut-dir /path/to/out_revised \  
4   --markers-out /path/to/out_revised/markers.json
```

- `--cut-dir` 用**新目录**，避免冲掉未备份的旧片。
-